

Adjustments to Union Lifetime Rules

Document #: P3726R0 [[Latest](#)] [[Status](#)]
Date: 2025-06-03
Project: Programming Language C++
Audience: CWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Tomasz Kamiński
<tomaszkam@gmail.com>

Contents

- [1 Introduction](#)
- [2 Proposal 1: Fixing When Implicit Lifetime Starts](#)
 - [2.1 Template-Argument-Equivalence](#)
 - [2.2 Wording](#)
- [3 Proposal 2: Fixing Which Values are Constituent Values](#)
- [4 Feature-Test Macro](#)
- [5 Acknowledgments](#)
- [6 References](#)

§ 1 Introduction

[[P3074R7](#)] ([trivial unions \(was std::uninitialized<T>\)](#)) was adopted in Hagenberg. One of the goals of that paper was to make an example like this work:

```
template <typename T, size_t N>
struct FixedVector {
    union { T storage[N]; };
    size_t size = 0;

    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(storage, storage + size);
    }

    constexpr auto push_back(T const& v) -> void {
        ::new (storage + size) T(v);
        ++size;
    }
};

constexpr auto silly_test() -> size_t {
```

```

FixedVector<std::string, 3> v;
v.push_back("some sufficiently longer string");
return v.size;
}

static_assert(silly_test() == 1);

```

That paper solved this problem by:

1. Making `unions` have trivial default constructors and trivial destructors, by default, and
2. Implicitly starting the lifetime of the first `union` member, if that member has implicit-lifetime type.

The first avoids pointless no-op empty constructors and destructors, and the second would start the lifetime of the `T[N]` member — which we need in order for the placement new to be well-defined. Which seemed to be a pretty nice thing, as the code just works, without any complicated intervention.

However, there is an important principle in C++ language design that Barry missed: we can't have nice things because there are no nice things. Richard Smith sent out this example:

```

union U { int a, b; };
template<U u> class X {};
constexpr U make() { U u; return u; }
void f(X<make(>)>) {}

```

He pointed out that today this is valid because template argument of `X` is a union object with no active member. But with the [\[P3074R7\]](#) changes, this:

1. Causes the union object to have an active member, and thus would have to be mangled differently. That change makes this an ABI break (although not all implementations mangle these cases differently, which is probably a bug, so the potential damage of ABI break isn't tremendous)
2. Causes the example to fail to compile because it's no longer a valid template argument.

The relevant rule here is 7.7 [\[expr.const\]](#) which says that:

- 22 A *constant expression* is either a glvalue core constant expression that refers to an object or a non-immediate function, or a prvalue core constant expression whose result object ([basic.lval]) satisfies the following constraints:
- (22.1) — each constituent reference refers to an object or a non-immediate function,
 - (22.2) — no constituent value of scalar type is an indeterminate or erroneous value ([basic.indet]),
 - (22.3) — [...]

where, in that same section:

2 The *constituent values* of an object *o* are

(2.1) — if *o* has scalar type, the value of *o*;

(2.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union members.

Note that inactive union members are excluded, but active-yet-uninitialized union members are disallowed.

This is also a problem because the original goal of the paper was to make types like `std::inplace_vector` completely usable at compile-time, and this rule (even separate from [\[P3074R7\]](#)) makes that impossible:

```
constexpr std::inplace_vector<int, 4> v = {1, 2};
```

A “normal” implementation would have `union { int storage[4]; }`, of which the first two elements are initialized but the last two are not. And thus have indeterminate value, so this isn’t a valid constant expression. For `int` specifically (and trivial types more broadly), this is fixable by having the implementation simply have a `int storage[4];` instead and initialize all the objects — since that’s free. But for types that either aren’t trivially default constructible or aren’t trivially destructible, that’s not an option, and that really shouldn’t be the limiting factor of whether you can create `constexpr` variables (or non-type template arguments) of such types.

We’re hoping to fix both of those issues in this paper, with two fairly independent fixes.

§ 2 Proposal 1: Fixing When Implicit Lifetime Starts

[\[P3074R7\]](#) added this wording to 11.4.5.2 [\[class.default.ctor\]](#):

4 If a default constructor of a union-like class *X* is trivial, then for each union *U* that is either *X* or an anonymous union member of *X*, if the first variant member, if any, of *U* has implicit-lifetime type ([basic.types.general]), the default constructor of *X* begins the lifetime of that member if it is not the active member of its union. [*Note 1*: It is already the active member if *U* was value-initialized. — end note] ~~An~~ Otherwise, an implicitly-defined ([dcl.fct.def.default]) default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no ctor-initializer ([class.base.init]) and an empty compound-statement.

That wording needs to be reverted. The default constructor will no longer start lifetimes implicitly.

Instead, we allow placement new on an aggregate element to start the lifetime of the aggregate. That is, given the above implementation:

```
template <typename T, size_t N>
struct FixedVector {
    union { T storage[N]; };
    size_t size = 0;

    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(storage, storage + size);
    }

    constexpr auto push_back(T const& v) -> void {
        ::new (storage + size) T(v);
        ++size;
    }
};

constexpr auto silly_test() -> size_t {
    FixedVector<std::string, 3> v;
    v.push_back("some sufficiently longer string");
    return v.size;
}

static_assert(silly_test() == 1);
```

This will work for the following reason:

[P3074R7]	This Paper
Default constructor starts lifetime of array (but not any elements)	Default constructor does not start any lifetime
The array is already within lifetime and is the active member	The act of placement-new onto the array starts the lifetime of the array and makes it the active member
Placement new is well-defined	Placement new is well-defined

We get to a well-defined state through a different route, but we still get to a well-defined state with reasonable code. Importantly, we don't change the behavior of existing code (as in Richard's example) since no lifetimes are implicitly created, and here we're allowing a placement new that is invalid today to instead also start lifetimes.

§ 2.1 Template-Argument-Equivalence

One of the consequences of the above proposal is what happens when we compare objects that should be equivalent but got there with different paths:

```

// see next section for making this work, but assume it does for now
constexpr auto v1 = FixedVector<int, 4>();

constexpr auto v2 = []{
    auto v = FixedVector<int, 4>();
    v.push_back(1);
    v.pop_back();
    return v;
}();

```

I didn't show `pop_back()` in the above implementation, but let's say it just does `storage[--size].~T()`. What can we say about `v1` and `v2`? Well, they're both empty vectors, so they compare equal. However, they're in different states:

- `v1`'s anonymous union has no active member, because we never started any lifetimes.
- `v2`'s anonymous union does have an active member `storage`, with no elements within lifetime.

Those wouldn't compare template-argument-equivalent, so `X<v1>` and `X<v2>` would be different types (for suitable template `X`). This isn't a very serious concern right now, since `FixedVector` isn't a structural type and that will remain true in C++26. But nevertheless, there is an easy way to ensure equivalence: by adding a new member to the union:

```

template <typename T, size_t N>
struct FixedVector {
    struct Empty { };
    union {
        Empty empty = {};
        T storage[N];
    };
    size_t size = 0;

    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(storage, storage + size);
    }

    constexpr auto push_back(T const& v) -> void {
        ::new (storage + size) T(v);
        ++size;
    }

    constexpr auto pop_back() -> void {
        storage[--size].~T();
        if (size == 0) {
            empty = Empty();
        }
    }
};

```

Now, v1 and v2 are in the same state: the active member of the union is empty.

§ 2.2 Wording

Revert the change in [class.default.ctor]/4:

- 4 ~~If a default constructor of a union-like class X is trivial, then for each union U that is either X or an anonymous union member of X , if the first variant member, if any, of U has implicit-lifetime type ([basic.types.general]), the default constructor of X begins the lifetime of that member if it is not the active member of its union. [*Note 1*: It is already the active member if U was value-initialized. — end note]~~ **An** Otherwise, an implicitly-defined ([dcl.fct.def.default]) default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no ctor-initializer ([class.base.init]) and an empty compound-statement.

Change 11.5.1 [class.union.general]/5:

- 5 When **either**
- (5.a) — the left operand of an assignment operator involves a member access expression ([expr.ref]) that nominates a union member **or**
- (5.b) — **the placement argument to a *new-expression* ([expr.new]) that is a non-allocating form ([new.delete.placement]) involves such a member access expression,**

it may begin the lifetime of that union member, as described below.

For an expression E , define the set $S(E)$ of subexpressions of E as follows:

- (5.1) — If E is of the form $A.B$, $S(E)$ contains the elements of $S(A)$, and also contains $A.B$ if B names a union member of a non-class, non-array type, or of a class type with a trivial default constructor that is not deleted, or an array of such types.
- (5.2) — If E is of the form $A[B]$ and is interpreted as a built-in array subscripting operator, $S(E)$ is $S(A)$ if A is of array type, $S(B)$ if B is of array type, and empty otherwise.
- (5.3) — Otherwise, $S(E)$ is empty.

In an assignment expression of the form $E1 = E2$ that uses either the built-in assignment operator ([expr.assign]) or a trivial assignment operator ([class.copy.assign]), for each element X of $S(E1)$ and each anonymous union member X ([class.union.anon]) that is a member of a union and has such an element as an immediate subobject (recursively), if modification of X would have undefined behavior under [basic.life], an object of the type of X is implicitly created in the nominated storage; no initialization is performed and the beginning of its lifetime is sequenced after the value computation of the left and right operands and before the assignment.

For an expression E , define the set $P(E)$ of subexpressions of E as follows:

- (5.4) — If E is of the form $\&A[B]$, E is interpreted as a built-in address operator, and $A[B]$ is interpreted as a built-in array subscripting operator, then $P(E)$ is A if A is of array type, B if B is of array type, and empty otherwise.
- (5.5) — If E has pointer type and is either
 - (5.5.1) — of the form $A + B$ and is interpreted as a built-in addition operator or
 - (5.5.2) — of the form $A - B$ and is interpreted as a built-in subtraction operator,then $P(E)$ is A if A is of array type, B if B is of array type, and the union of $P(A)$ and $P(B)$ otherwise.
- (5.6) — Otherwise, $P(E)$ is empty.

In a *new-expression* with a *new-placement* of the form (E) that uses a non-allocating form (`[new.delete.placement]`), for each element X of $P(E)$ that names a union member and each anonymous union member X that is a member of a union and has such an element as an immediate subobject (recursively), if X is not within its lifetime, the lifetime of an object of the type of X is started in the nominated storage; no subobjects are created and the beginning of its lifetime is sequenced immediately before the value computation of E .

[*Note 2*: This ends the lifetime of the previously-active member of the union, if any (`[basic.life]`). — *end note*]

§ 3 Proposal 2: Fixing Which Values are Constituent Values

The current rule for constituent values is, from 7.7 [\[expr.const\]/2](#):

2 The *constituent values* of an object o are

- (2.1) — if o has scalar type, the value of o ;
- (2.2) — otherwise, the constituent values of any direct subobjects of o other than inactive union members.

As mentioned earlier, this means that if we have a `union { T storage[4]; }` then either there are no constituent values (if `storage` is inactive) or we consider all of the `T`s as constituent values (even if we only constructed the first two). So we'll need to loosen this rule to permit objects with union members to be more usable as constant expressions.

For the `FixedVector` (aka `static_vector` aka `inplace_vector`) example, we really only need to allow “holes” at the end of the array. But if we want to support a different container, that is more bidirectional and supports cheap `push_front` and `pop_front`, we will also want to support “holes” at the front of the array. So for simplicity, we're proposing to support holes *anywhere* in the array. Note that we're still not

proposing nice syntax for actually constructing such an array with holes. Richard on the reflector had suggested a strawperson syntax:

```
// short array initializer:
// initializes arr[0] and arr[1],
// does not start lifetime of rest
int arr[42] = {a, b, short};

// in std::allocator<T>::allocate:
return new (ptr) T[n]{short};
```

I don't think we strictly need to solve that problem right now, but at least we can put in the groundwork for supporting it in the future.

Until then, we're proposing something like this change to 7.7 [\[expr.const\]](#):

2 The *constituent values* of an object *o* are

- (2.1) — if *o* has scalar type, the value of *o*;
- (2.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union ~~members~~ subobjects (see below).

The *constituent references* of an object *o* are

- (2.3) — any direct members of *o* that have reference type, and
- (2.4) — the constituent references of any direct subobjects of *o* other than inactive union ~~members~~ subobjects.

An *inactive union subobject* is either:

- (2.5) — an inactive union member or
- (2.6) — an element *A* of an array member of a union where *A* is not within its lifetime.

[Example 1:

```
struct A {
    struct X {
        int i;
        int j;
    };

    struct Y {
        X x1;
        X x2;
    };

    union {
        int i;
        int arr[4];
    };
};
```



```

        Y y;
    };
};

constexpr A v1;           // ok, no constituent values
constexpr A v2{.i=1};    // ok, the constituent values are {v2.i}
constexpr A v3 = []{
    A a;
    new (&a.arr[1]) int(1);
    new (&a.arr[2]) int(2);
    return a;
}();                      // ok, the constituent values are {v3.arr[1],
                          v3.arr[2]}
constexpr A v4 = []{
    A a;
    a.y.x1.i = 1;
    a.y.x2.j = 2;
    return a;
}();                      // error: the constituent values include v4.y.x1.j
                          and v4.y.x2.i
//                          // which have erroneous value

— end example ]

```

And extend the template-argument-equivalent rules to understand this, in 13.6 [\[temp.type\]](#):

- 2 Two values are *template-argument-equivalent* if they are of the same type and
 - (2.1) — [...]
 - (2.8) — they are of array type and their corresponding elements are **either both within lifetime and** template-argument-equivalent **or both not within their lifetime**, or
 - (2.9) — [...]

That fix ensures that:

```
constexpr std::inplace_vector<int, 4> v = {1, 2};
```

is a valid constexpr variable if the implementation uses a `union { int storage[4]; }` to hold the data, because we would only consider the first two elements of `storage` as constituent values — the fact that the last two elements are uninitialized no longer counts against us when we consider whether `v` is a valid result of a constant expression.

§ 4 Feature-Test Macro

And bump the feature-test macro added by [\[P3074R7\]](#):

```
- __cpp_trivial_union 202502L  
+ __cpp_trivial_union 2025XXL
```

§ 5 Acknowledgments

Thank you to Richard Smith for bringing the issue to our attention and for all the helpful suggestions.

§ 6 References

[P3074R7] Barry Revzin. 2025-02-14. trivial `unions` (was `std::uninitialized<T>`).
<https://wg21.link/p3074r7>